



Informe de uso de Git y Github en el programa Extractor de significados de textos

Gonzalo Albornoz

Introducción a los Repositorios de Código Distribuido

Profesor de cátedra: Felipe Gómez

1. Descripción del proyecto

El proyecto consiste en un programa con interfaz gráfica que extrae resúmenes o clasificaciones de una gran cantidad de filas de una columna específica de un archivo Excel o CSV, utilizando un LLM a través de la API de Ollama.

El código base de este programa fue hecho durante mi práctica profesional, antes del inicio del semestre. El trabajo realizado durante esta tarea no consistió en escribir el programa de nuevo, sino en crear nuevas funciones para este y aplicar este trabajo completo en Git (ramas, integración de cambios y resolución de conflictos), que es el objetivo de la evaluación. Teniendo en cuenta esto, también se utilizó la inteligencia artificial para escritura del código y sus cambios.

Problema abordado: el programa original no permitía la extracción de subconjuntos específicos de datos, ni visualizar en forma de gráfico la distribución de resultados. Durante la tarea se agregaron tres funciones nuevas para resolver esto:

- Filtro por columna y valor (búsqueda avanzada): Extraer solo las filas donde una columna del archivo coincide con un valor específico.
- Filtro por rango de filas: Extracción de una parte específica del archivo (por ejemplo, de la fila 4 a la 32), de forma independiente o con el filtro anterior.
- Gráfico de resultados: una casilla opcional que al marcarla, se genera un gráfico de barras de la extracción de significados de la tabla.

Estas tres funciones se desarrollaron en una rama diferente de Git, y su integración al main se hizo con merge.

2. Estructura del repositorio

El repositorio contiene los siguientes archivos principales:

- `Extractor_de_significado_en_textos.py` — Código fuente completo de la aplicación.
- `README.md` — Documentación de uso: Funcionamiento del programa y las funciones agregadas.
- `.gitignore` — Excluye archivos `“venv/”`, `“*.exe”` y archivos de configuración generados por el programa (`.clasificador_config.json`).
- `Informe.pdf` — Este informe.

3. Evolución del desarrollo

El desarrollo se hizo en la rama principal (main) y tres ramas de desarrollo creadas específicamente para este período de evaluación:

- `nueva_funcion` — Creación del filtro por columna y valor (búsqueda avanzada).

- nueva_funcion_2 — Creación del filtro por rango de filas.
- grafico — Creación del gráfico de barras de resultados.

Cronología resumida:

- Se inicializó el repositorio local con el código base existente, y se subió a GitHub.
- En la rama nueva_funcion se agregó la función de filtrado, luego el elemento de interfaz (Combos sueltos, pero al probarlos los rediseñe como un popup de "Búsqueda avanzada" debido a un fallo en la visualización).
- Se hizo merge de nueva_funcion a main.
- Desde main se creó la rama nueva_funcion_2 (Filtro por rango) y cambios directos sobre main.
- Al hacer merge de nueva_funcion_2 con main, ambas ramas habían modificado el mismo bloque de código (dentro de _aplicar), lo que generó un conflicto de merge, este se resolvió manualmente.
- Se desarrolló la rama grafico de forma independiente y se hizo merge a main.

4. Análisis del historial de Git

a. Número total de commits y días con actividad

El repositorio registra actividad en 2 días distintos (3 y 4 de julio), con un total de 14 commits repartidos entre la rama principal y tres ramas de desarrollo.

```
gonzalo@gonzalo-Latitude-E5440:~/Extractor de significados$ git log --pretty=format:"%h | %ad | %an | %s" --date=short
cc4f332 | 2026-07-04 | Vear511 | README actualizado
4eae316 | 2026-07-04 | Vear511 | Merge grafico
1ec229b | 2026-07-04 | Vear511 | Agrega generacion de grafico
d8db2bd | 2026-07-04 | Vear511 | Merge conflict artificial solucionado
997d680 | 2026-07-04 | Vear511 | 2da preparacion merge conflict
9d680a5 | 2026-07-04 | Vear511 | Preparacion mege conflic artificial
bd65c1d | 2026-07-04 | Vear511 | Bug filtrar_por_rango
3df69f3 | 2026-07-04 | Vear511 | Agrega filtrar_por_rango en popup
6592e74 | 2026-07-04 | Vear511 | Agrega funcion filtrar_por_rango
cf1e06e | 2026-07-04 | Vear511 | Merge nueva_funcion
32f8bfb | 2026-07-04 | Vear511 | Agrega filtro por columna
fba7f2d | 2026-07-03 | Vear511 | Agrega funcion priorizar
25f9489 | 2026-07-03 | Vear511 | Agrega README
c68f308 | 2026-07-03 | Vear511 | Commit inicial)
gonzalo@gonzalo-Latitude-E5440:~/Extractor de significados$
```

Nota: El desarrollo comenzó tarde dentro del período habilitado (24 de junio al 5 de julio) debido a que priorice el estudio para otras evaluaciones.

b. Archivos más modificados

Al tratarse de un proyecto de un único archivo fuente, “Extractor_de_significado_en_textos.py”, en este concentra casi todos los cambios de código, esto debido a que se tiene pensado convertir el código en un ejecutable por lo tanto se concentra todo esté en un solo archivo, tal cual como se

hizo con el código base. README.md y .gitignore recibieron cambios de documentación y configuración.

c. Commits más relevantes

- Commit inicial — Código base hecho en la práctica profesional.
- Agrega filtro por columna — Primera función nueva.
- Merge nueva_funcion — Primer merge.
- Agrega funcion filtrar_por_rango y Bug filtrar_por_rango — Desarrollo y corrección de la segunda función.
- Merge nueva_funcion_2 en main (merge conflict) — Commit central de este informe.
- Merge gráfico en main — Creación de la tercera función.

d. Ramas utilizadas

- main (rama principal)
- nueva_funcion
- nueva_funcion_2
- grafico

e. Merges realizados

Se realizaron 3 merges mediante git merge --no-ff, lo que asegura que cada integración quede registrada como un commit explícito de dos padres en el historial.

f. Evidencia del three-way merge

El merge conflict tiene dos padres directos, uno perteneciente a main y otro a nueva_funcion_2, cumpliendo la definición de three-way merge.

```
* cc4f332 (HEAD -> main, origin/main) README actualizado
* 4eae316 Merge grafico
| \
| * 1ec229b (origin/grafico, grafico) Agrega generacion de grafico
| /
| * d8db2bd Merge conflict artificial solucionado
| \
| * bd65c1d (origin/nueva_funcion_2, nueva_funcion_2) Bug filtrar_por_rango
| * 3df69f3 Agrega filtrar_por_rango en popup
| * 6592e74 Agrega funcion filtrar_por_rango
| * | 997d680 2da preparacion merge conflict
| * | 9d680a5 Preparacion mege conflic artificial
| /
| * cf1e06e Merge nueva_funcion
| \
| * 32f8bfb (origin/nueva_funcion, nueva_funcion) Agrega filtro por columna
| * fba7f2d Agrega funcion priorizar
| /
* 25f9489 Agrega README
* c68f308 Commit inicial
```

g. Evidencia y explicación de la resolución manual de conflictos

Se produjo un conflicto de merge intencional en main y nueva_funcion_2, se modificaron de forma distinta el mismo bloque de código (_aplicar). Al intentar fusionarlas, Git no supo qué cambios priorizar, por lo que detuvo el proceso y marcó el archivo.

5. Análisis de integración de cambios

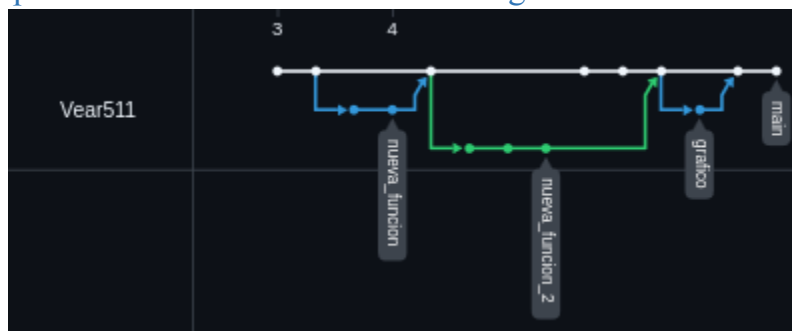
a. Descripción de las ramas utilizadas

- nueva_funcion: desarrollo del filtro por columna y valor y su interfaz (popup de búsqueda avanzada).
- nueva_funcion_2: desarrollo del filtro por rango de filas, integrado en el popup.
- grafico: Generación opcional del gráfico de barras a travez de una casilla.

b. Objetivo de cada rama

Las ramas se crearon para el desarrollo de cada funcionalidad separada, permitiendo probarla y corregirla sin afectar el main hasta que estuviera lista para integrarse.

c. Captura del historial mostrando el merge



d. Explicación del conflicto generado

El conflicto lo generé a propósito para cumplir con lo que pide la pauta de generar y resolver un conflicto de forma manual. Para esto, edité main y nueva_funcion_2 al mismo tiempo. En main cambié directamente el bloque que arma el texto de estado del filtro (dentro de _aplicar), agregando un emoji y un mensaje diferente. En nueva_funcion_2, reescribí ese mismo bloque para que también mostrará el rango de filas. Al hacer git merge --no-ff nueva_funcion_2 estando en main, Git no pudo combinar las dos versiones de ese bloque, porque ambas cambiaban las mismas líneas, y marcó el archivo como "modificado por ambos", deteniendo el merge para que yo lo resolviera a mano.

e. Explicación de la resolución manual aplicada

Para resolver el conflicto terminé combinando ambas versiones. Mantuve lo del rango de filas y el mensaje del filtro nuevo. Después de resolverlo esto hice git add y git commit para cerrar el merge, y probé el programa para confirmar que las dos funciones (filtro por columna y rango de filas) funcionaban juntas sin problema.

6. Visualización del repositorio

A continuación se incluyen capturas del historial del repositorio en GitHub.

README actualizado	cc4f332	<>
Vear511 committed 20 hours ago		
Merge grafico	4eae316	<>
Vear511 committed 20 hours ago		
Agrega generacion de grafico	1ec229b	<>
Vear511 committed yesterday		
Merge conflict artificial solucionado	d8db2bd	<>
Vear511 committed yesterday		
2da preparacion merge conflict	997d680	<>
Vear511 committed yesterday		
Preparacion mege conflic artificial	9d680a5	<>
Vear511 committed yesterday		
Bug filtrar_por_rango	bd65c1d	<>
Vear511 committed yesterday		
Agrega filtrar_por_rango en popup	3df69f3	<>
Vear511 committed yesterday		
Agrega funcion filtrar_por_rango	6592e74	<>
Vear511 committed yesterday		
Merge nueva_funcion	cf1e06e	<>
Vear511 committed 2 days ago		
Agrega filtro por columna	32f8bfb	<>
Vear511 committed 2 days ago		
Commits on Jul 3, 2026		
Agrega funcion priorizar	fb37f2d	<>
Vear511 committed 2 days ago		
Agrega README	25f9489	<>
Vear511 committed 2 days ago		
Commit inicial)	c68f308	<>
Vear511 committed 2 days ago		

7. Análisis de métricas

- Commits por día: La actividad se hizo en 2 días (3 y 4 de julio), con mayor commits en el segundo día.
- Archivos modificados por commit: La mayoría de los commits modifican un archivo (el script principal), lo cual es consistente con un proyecto de un solo módulo.
- Ramas creadas y fusionadas: 3 de 3 ramas de desarrollo fueron mergeadas exitosamente a main, dos de forma limpia y una con conflicto.
- Cantidad de merges: 3 merges realizados con --no-ff, uno de ellos con conflicto real.

8. Reflexión final

Este proyecto me ayudó en entender la creación de ramas para la modificación exitosa de nuevas funcionalidades para código ya creados, esto debido a que el código base de este proyecto ya servía de manera correcta, pero podía ser mejorado para una mejor experiencia de usuario.

El conflict merge artificial, me resulto util para entender cómo es posible resolver problemas parecidos que otros proyectos puedan tener, pero no solo eso, sino que al intentar crear este conflicto, hize un merge exitoso de las dos ramas, teniendo en cuenta que modifique las dos ramas por separado, pero debido a que no modifique bloques de código diferentes, este merge fue exitoso gracias a Git.

Por lo tanto, siento que este proyecto me ayudó a ampliar mi conocimiento de creación de repositorios y la edición de estos.